Intelligent Network Configuration Assistant

Simplifying Network Management Through AI, MCP and Rule-Based Automation

INA v2: Policy-Guarded Natural-Language Network Configuration on Raspberry Pi 5 | Khushi Garg 24110164 • Dhruv Meshram 24110109 • Ananya Patel 24110038 • Vedant Bondekar 24110384

1. Manual Configuration

- Administrators use terminal commands and Linux networking utilities
- Needs knowledge of firewall rules, interfaces, routing and traffic control

2. Distributed Management

- Discovery, access control and bandwidth use different utilities
- No single natural-language interface to coordinate them

3. AI Automation Risk

- Direct LLM access to system commands can cause unsafe changes
- Natural language must not become privileged operations

Our solution: four cooperating parts

AI Assistant understands requests → **Policy Engine validates authorization** → **MCP standard tool link** → **Raspberry Pi Router configures network**

Before and after

Traditional

Administrator → **Terminal** → **Linux Commands** → **Router**

INA

Administrator → **Natural Language** → **AI + Policy** → **MCP** → **Router**

Example interaction

```
Admin> Block client 192.168.50.46.
INA> Request interpreted.
INA> Authorization successful.
INA> Client successfully blocked.
```

AI proposes. Policy decides. Network verifies.

Distributed Architecture: Laptop Meets Raspberry Pi 5

Control plane on the laptop, network/data plane on the Raspberry Pi

LAPTOP: CONTROL PLANE

Administrator Interface (CLI)

- Admin enters commands
- Shows devices, status, results

AI Assistant

- Parses request, picks tool
- Claude, Gemini, OpenRouter models
-

Rule Based Parser (Alternative)

- Incase the LLM doesn't work, rule based execution and interpretation of commands happen.

MCP Client

- Discovers remote tools
- Sends structured requests
- Receives results

MCP tool calls

Streamable HTTP over HTTPS / TLS 1.3

RASPBERRY PI 5: NETWORK / DATA PLANE

MCP Server (FastMCP)

- Eight typed tools
- Pydantic v2 argument validation

Network Config Layer

- Discovery, access control
- Bandwidth, client isolation
- Network verification

Policy Engine and privilege

- Separates unprivileged MCP from privileged ops
- Restricted sudo command
- Roles and authorization
- Applies network rules
- Rejects invalid; asks for missing info

Linux Networking

- nftables; tc HTB + IFB
- hostapd; dnsmasq
- IP forwarding and NAT

Phone hotspot (internet) → **Pi wlan0 (uplink)** → **Pi ap0, SSID INA-Router 192.168.50.1/24, DHCP .10 to .100** → **Connected clients (incl. laptop)**

The laptop handles intelligence and authorization, while the Raspberry Pi performs actual network enforcement.

From Natural Language to Real Network Configuration

Example: "Limit client 192.168.50.46 to 5 Mbps."

1 Interpretation

- CLI submits request
- AI identifies operation
- Extracts IP and 5 Mbps
- Selects MCP tool

2 Authorization

- Identity and role checked
- Policy rules applied
- Protected devices enforced
- Invalid requests rejected

3 Communication

- Client invokes tool
- Streamable HTTP over HTTPS
- Pi MCP server receives
- Pydantic validates args

4 Enforcement

- Network module invoked
- Restricted privileged service
- tc applies the limit
- nftables rules as needed

5 Verification

- Actual state checked
- Rollback on failure
- Audit log entry
- Result back to laptop

Software modules

Module	Responsibility
`assistant/`	AI interaction and MCP client
`policy/`	Authorization and rule evaluation
`mcp_server/`	MCP server and tool definitions
`network/`	Discovery, access control, bandwidth, verification
`config/`	System and policy configuration
`tests/`	Unit, integration and validation tests

Technologies

Python	FastMCP	MCP
Pydantic v2	Streamable HTTP	HTTPS/TLS
nftables	tc	IFB
hostapd	dnsmasq	systemd

Secure by Design: Controlled AI-Driven Network Automation

Trust boundaries, cooperating safeguards and practical network engineering

UNTRUSTED: AI Assistant

- Interprets, proposes tool calls
- Cannot run shell commands
- Cannot override authorization

TRUSTED: Policy and Enforcement

Policy Authorization → Restricted Execution → Verification → Audit Log

The LLM never receives unrestricted root access.

Role-Based Access

Admin, operator, viewer roles decide allowed operations.

Device-Bound Authorization

Permissions tied to registered users and devices; critical devices protected.

Typed Input Validation

Pydantic v2 schemas reject malformed or invalid parameters.

Privilege Separation

MCP server has no unrestricted root; one restricted sudo boundary.

Prompt Injection Resistance

Model text cannot override policy; execution is controlled independently.

Audit and Recovery

Request IDs, hash-chained records, state verified, rollback on failure.

Engineering Beyond Basic Configuration

AP startup reliability

systemd ordering of AP interface, hostapd, dnsmasq

Client isolation

AP isolation + proxy ARP private VLAN setup

Download shaping

IFB redirects ingress so tc can shape it

Firewall rule order

Block rules precede established-connection handling

A Functional Network Management System

Implemented capabilities and observed behavior

Client Discovery

Illustrative example, not a live snapshot

Client	IP Address	State
Client 1	192.168.50.46	Connected
Client 2	192.168.50.51	Connected
Client 3	192.168.50.62	Connected

Access Control

- Block and unblock a client
- Restrict internet; control peer traffic
- Isolate a client from other devices

Bandwidth Management

- Per-client tc HTB limits; IFB for downloads

Network Verification

- Checks real behavior, not just command success
- Client reachability
- Internet access
- Peer communication
- Bandwidth behavior
- Restoration after unblock

Bandwidth card (illustrative)

Client: 192.168.50.46
Configured limit: 5 Mbps
Status: Applied

Testing scope

- Raspberry Pi 5 router: ping and TCP throughput measurements
- Linux network-namespace testbed validated enforcement behavior (separate from Pi hardware tests)
- End-to-end laptop-to-Pi MCP communication

16/16
functional tests passed

10/10
clarification tests passed

0 / 65
unsafe executions in adversarial runs

25.2 Mbps
local TCP throughput

0%
client-to-router packet loss

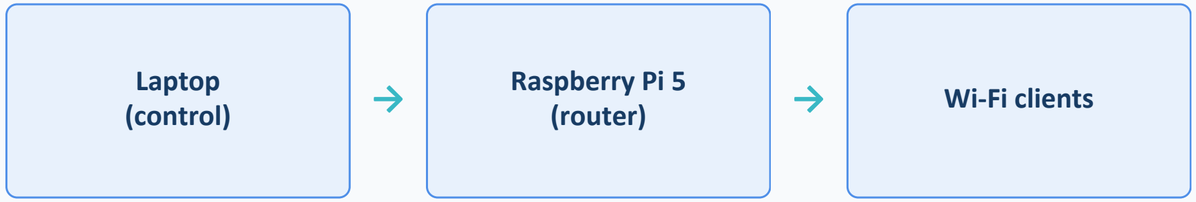
Making Network Management More Accessible and Secure

What we achieved

- Natural-language interface for network administration
- AI assistance integrated with MCP tool communication
- Rule-based policy engine for authorization
- Deployed on a Raspberry Pi 5 router
- Real firewall and bandwidth enforcement
- Client discovery, access control, verification
- AI interpretation separated from privileged execution

Why the Raspberry Pi matters

- Not a simulated network: the Pi is an actual Wi-Fi router
- Restrictions enforced at the Linux networking layer



Future scope

- **Web-based admin dashboard**
- **Multi-router and multi-AP management**
- **IPv6 support**
- **Advanced monitoring and analytics**
- **Adaptive policies from network conditions**

AI proposes. Policy decides. Network verifies.

Khushi Garg • Dhruv Meshram • Ananya Patel • Vedant Bondekar | Team T009 | Thank You